Lux: Technical Architecture Specification

Visual Output Surface for AI Agents

May 2026 — v0.18.0

Contents

1 Overview

Lux is a visual output surface for Claude Code. It renders element trees pushed by AI agents via MCP tools into a native ImGui window over Unix socket IPC. The system has four projection surfaces from a single codebase: Python library, CLI, MCP server, and Claude Code plugin.

The display server owns the render loop. Agents are clients. All agent–display communication is message-passing over a framed JSON protocol on a Unix domain socket. The display server is single-threaded: no threads, no asyncio, no locks. It polls all I/O non-blocking each frame via `select()`.

1.1 Companion Documents

This document is the canonical, comprehensive description of the shipped system. The sibling files in `docs/architecture/` are narrower slices and design proposals that this document references but does not duplicate:

- `domain-model.md` — north-star domain (Element, Scene, Client, Update, Event, Port). *Target state*, not yet implemented. Section §12 flags where the current procedural shape diverges from the OO target.
- `x11-model.md` — three-process target architecture (mcp-proxy → `luxd` → `lux-display`). *Proposed*; the shipped code runs the display and hub in one process. Section §6 describes the current single-process reality and the boundary that will become a process split.
- `luxd-impl.md` — Phase 1 implementation spec for the three-process split.
- `introspection-api.md` — the unified `QueryRequest`/`QueryResponse` envelope. The generic infrastructure is shipped; twelve of fourteen planned query methods are wired and live — a thirteenth (`screenshot`) is registered but its handler raises and routes to a dedicated `ScreenshotRequest` message instead. See Section §10.
- `display-server.tex` — Z specification of the display state machine, with refinement and partition tests (Section §11).

1.2 Design Principles

P1. Composable element tree. Every element kind is small, nestable, and data-driven. If an agent can describe it as JSON, Lux renders it.

P2. Protocol is the API surface. The wire protocol is the only interface between agents and the display. No shared memory, no function calls, no Python imports required.

P3. Single-threaded simplicity. The display server uses a polling event loop with zero threads. All state is owned by the main thread. No synchronization primitives.

P4. Incremental by default. Scenes can be patched by element ID without replacing the entire tree. Table filters run at frame rate with zero MCP round-trips.

P5. Multi-scene coexistence. Each `show()` call opens a new tab. Scenes persist until explicitly dismissed or cleared. Same `scene_id` replaces content in-place.

2 System Architecture

2.1 Component Map

The Python package is laid out as five domain-aligned packages plus a small set of foundation modules at the package root.

Module	File or package	Responsibility
Protocol	`src/punt_lux/protocol/`	Wire framing (4-byte length prefix, JSON payload), the `FrameReader` streaming buffer, and the dispatch helpers `message_to_dict` / `element_to_dict`. Element and message dataclasses live in the two sub-packages below.
Elements	`src/punt_lux/protocol/elements/`	24 element dataclasses across seven family modules (`basics`, `inputs`, `layout`, `graphics`, `table`, `patch`, `plot_element`), with the `ElementCodec` dispatch table in `codec.py` (PR #169, #172). Draw commands are split into `draw_wire.py`, `draw_values.py`, `draw_bounds.py`, `draw_decoder.py`, and four command-family modules (`draw_commands_line.py`, `draw_commands_shape.py`, `draw_commands_curve.py`, `draw_commands_text.py`) (PR #176, #177).
Messages	`src/punt_lux/protocol/messages/`	21 message dataclasses across five family modules (`lifecycle`, `scene`, `interaction`, `menu`, `introspect`), with the `MessageRegistry` dispatch table in `registry.py` (PR #157, #170).
Display	`src/punt_lux/display/`	ImGui render loop, element renderers, event flush. Six modules: `server.py` (∼1,400 lines — the `DisplayServer` coordinator), `element_renderer.py` (∼1,100 lines — per-kind dispatch with a 24-entry `_RENDERERS` table), `table_renderer.py` (∼630 lines), `menu_manager.py` (∼580 lines), `idle_screen.py` (∼200 lines), `texture_cache.py` (∼80 lines). Extracted from the original `display.py` (4,208 lines) via PR #158; `server.py` is the remaining decomposition target.
Scene	`src/punt_lux/scene/`	Scene graph state. `manager.py` owns the `SceneManager` state machine (∼440 lines). `frame.py` defines the `Frame` class (∼140 lines; class renamed from `_Frame` in PR #159). `widget_state.py` defines `WidgetState` (36 lines).
Tools (MCP)	`src/punt_lux/tools/`	FastMCP tool surface (PR #160). `server.py` owns the `FastMCP` instance and the session lifespan. `connection.py` owns `_get_client` and `_with_reconnect`. `tools.py` registers the 24 tool functions (Section §10).
Socket server	`src/punt_lux/socket_server.py`	`SocketServer` — Unix-domain socket lifecycle, non-blocking multiplexing of client connections, per-client `FrameReader` buffers, send/recv to clients. Extracted from the original display monolith.
Query dispatcher	`src/punt_lux/query_dispatcher.py`	`QueryDispatcher` — routes `QueryRequest` by `method` string to registered handler methods. Owns the ring buffers for recent interaction events (max 200) and recent error log entries (max 100). See `introspection-api.md` for the wire format and handler template.
Client	`src/punt_lux/client.py`	Backward-compat shim that re-exports `DisplayClient` under the old name `LuxClient`. Scheduled for removal (`lux-3bp8`, per PL-PP-1: no compat aliases).
Display client	`src/punt_lux/display_client.py`	`DisplayClient` — high-level client with a background listener thread, typed methods for

2.2 Data Flow

Three primary data paths connect agents to the display:

2.2.1 Write Path (Agent → Display)

1. Agent calls MCP tool (e.g., `show()`).
2. MCP server obtains a `DisplayClient` via `tools/connection.py::_get_client()`, calls `client.show(scene_id, elements)`.
3. Client serializes `SceneMessage` as JSON, frames with 4-byte big-endian length prefix, sends over Unix socket.
4. `SocketServer` (`src/punt_lux/socket_server.py`) receives the frame in `poll_clients()`, deserializes via `FrameReader.drain_typed()`, dispatches to `DisplayServer._handle_message()`.
5. `DisplayServer` delegates state updates to `SceneManager.handle_scene()` or `handle_framed_scene()`.
6. `DisplayServer._render_scene()` draws the active scene each frame via `ElementRenderer`.
7. `DisplayServer` sends an `AckMessage` back to the client.

2.2.2 Event Path (User → Agent)

1. User interacts with widget (click, slider drag, text input).
2. `ElementRenderer` or `TableRenderer` emits an `InteractionMessage` with `element_id`, `action`, `value`, and optional `scene_id` via the `emit_event` callback.
3. Message buffered in `DisplayServer._event_queue` and pushed to the `QueryDispatcher`'s ring buffer for later `list_recent_events()` queries.
4. Each frame, `_flush_events()` broadcasts all queued messages to all connected clients via `SocketServer.send_to_cl` then clears the queue.
5. Agent calls `recv()` to consume events.

2.2.3 Update Path (Incremental Patching)

1. Agent calls `update(scene_id, patches)`.
2. `DisplayServer` receives `UpdateMessage`, delegates to `SceneManager.apply_update()`.
3. `_find_element()` (module helper in `scene/manager.py`) locates target by ID via depth-first search through the element tree.
4. Patch applied: `set` updates fields in-place, `remove` deletes element and cleans up widget state via `WidgetState.set(eid, None)` and `clear_suffix()`.
5. Routes to correct scene by `scene_id` (not limited to active tab); `SceneManager.resolve_scene()` looks up either the unframed-scenes dict or the per-frame scene dict.

3 Wire Protocol

3.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----------------+---------------------------+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----------------+---------------------------+
```

Maximum payload size: 16 MiB (2^{24} bytes). The `FrameReader` (in `src/punt_lux/protocol/__init__.py`) accumulates partial reads into a byte buffer and yields complete frames via `drain()` (raw dicts) or `drain_typed()` (Message dataclasses). Oversize frames raise `ValueError`, which `SocketServer` converts to a client disconnect.

3.2 Message Types

The 21 message types are split across two unions in `src/punt_lux/protocol/messages/__init__.py`: `ClientMessage` (12 types) and `DisplayMessage` (8 types), plus `UnknownMessage` for forward compatibility.

3.2.1 Client → Display

Type	Purpose
`SceneMessage`	Replace or add scene. Fields: `id`, `elements[]`, `title`, `layout`, `frame_id`, `frame_title`, `frame_size`, `frame_flags`, `frame_layout`.
`UpdateMessage`	Incremental patches. Fields: `scene_id`, `patches[]` (each: `id`, `set`, `remove`).
`ClearMessage`	Remove all scenes and reset state.
`PingMessage`	Heartbeat. Field: `ts`.
`ConnectMessage`	Client identity handshake. Field: `name`.
`MenuMessage`	Set agent-provided menu entries.
`ThemeMessage`	Apply theme by name.
`RegisterMenuMessage`	Register per-client menu items. Field: `items[]`.
`IntrospectRequest`	Request scene element tree (legacy ad-hoc op; retained alongside `QueryRequest` for backward compatibility). Field: `scene_id`.
`ListScenesRequest`	Request list of all active scenes (legacy ad-hoc op).
`ScreenshotRequest`	Request display screenshot (legacy ad-hoc op).
`QueryRequest`	Generic introspection/control request. Fields: `method`, `params`. The dispatcher routes by `method` string — see `introspection-api.md`.

SceneMessage frame fields (since v0.9.0): `SceneMessage` includes optional fields `frame_id`, `frame_title`, `frame_size` (initial `[width, height]`), `frame_flags` (dict of ImGui window flags such as `no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`), and `frame_layout` (`"tab"` or `"stack"`).

3.2.2 Display → Client

Type	Purpose
`ReadyMessage`	Handshake on connect; protocol version, capabilities.
`AckMessage`	Acknowledges scene or update; `scene_id`, `ts`, optional `error`.
`InteractionMessage`	User interaction; `element_id`, `action`, `value`, `ts`, optional `scene_id`.
`PongMessage`	Ping response; round-trip latency probe.
`IntrospectResponse`	Scene element tree response.
`ListScenesResponse`	List of active scene summaries plus frame summaries (`scenes[]`, `frames[]`).
`ScreenshotResponse`	Path to captured PNG.
`QueryResponse`	Generic introspection/control response. Fields: `method`, `result`, optional `error`.

Unknown message types are deserialized as `UnknownMessage` for forward compatibility rather than disconnecting the client.

The dispatch table (`MessageRegistry` in `src/punt_lux/protocol/messages/registry.py`) is populated at import time by `register_codecs` calls from each family module. Tests can construct an isolated registry without touching the module-level default — this replaces the original `if/elif` chain in `message_from_dict` (PR #157).

3.3 Element Kinds

24 element kinds organized by category, registered in `ElementCodec` via per-family `register_codecs` calls. Each element is a frozen dataclass with `slots=True`. The `Element` union is assembled in `src/punt_lux/protocol/elements/__init__.py`.

Family module	Element	Notes
`basics`	`text`	Content + style (heading, caption, success, error, code); optional `color` hex string
	`markdown`	CommonMark rendering via imgui_md
	`image`	File path or base64 data
	`separator`	Visual divider
	`progress`	Fraction bar with label
	`spinner`	Loading indicator
`inputs`	`button`	Label, action, disabled flag; `arrow` (directional) and `small` variants
	`slider`	Min/max/value, integer mode
	`checkbox`	Boolean toggle
	`combo`	Dropdown selection
	`input_text`	Single-line text input
	`radio`	Radio button group
	`input_number`	Numeric input with step buttons, min/max clamping, integer mode
	`color_picker`	RGB/RGBA hex picker; `alpha` (RGBA) and `picker` (full widget) modes
	`selectable`	Click-to-select item
`layout`	`group`	Row, column, or paged layout (`pages` + `page_source`)
	`tab_bar`	Tabbed container
	`collapsing_header`	Expandable section
	`window`	Movable, resizable floating panel
	`tree`	Recursive node hierarchy; `flat` flag for non-indented expansion
	`modal`	Modal popup dialog; container with children, emits `"closed"` event
`graphics`	`draw`	Canvas with typed draw commands (see §3.4)
	`plot`	ImPlot: line, scatter, bar series
`table`	`table`	Columns, rows, built-in filters, detail panel, pagination

3.4 Draw Commands

The `draw` element holds a tuple of typed `DrawCommand` records (PR #176, #177). Each command is a frozen, slotted dataclass that satisfies the `DrawCommand` Protocol (`TYPE: ClassVar[str]`, `to_wire(self) -> dict`, `from_wire(cls, d, *, ctx) -> Self`).

Module	Record	Geometry
draw_commands_line	Line	Start, end, color, thickness.
	Polyline	Point list, color, thickness, closed flag.
draw_commands_shape	Rect	Top-left, bottom-right, color, thickness, rounding, filled.
	Circle	Center, radius, color, thickness, filled.
	Triangle	Three points, color, thickness, filled.
draw_commands_curve	BezierCubic	Four control points, color, thickness.
draw_commands_text	TextGlyph	Position, content, color.

Shared value classes live in `draw_values.py` and `draw_bounds.py`: `Point2` (validated $[x, y]$ pair), `Color` (`#RRGGBB` / `#RRGGBBAA` hex), `Thickness` (strictly positive stroke width), `Radius` (strictly positive), `Rounding` (non-negative). The `WireContext` in `draw_wire.py` carries the per-error prefix (`draw command [i] (kind)`) and the require-or-raise helpers (`require_number`, `require_sequence`, etc.). `DrawCommandDecoder` in `draw_decoder.py` dispatches on the wire `cmd` string to the appropriate `from_wire` classmethod; `DrawElement.commands` is typed `tuple[DrawCommand, ...]` and the renderer dispatches via a typed `match` statement.

4 Display Server Internals

4.1 Frame Cycle

The display server runs an ImGui render loop via `hello_imgui.run()`. Each frame, the `DisplayServer._on_frame()` callback executes four phases in sequence:

1. **Accept connections.** `SocketServer.accept_connections()` runs `select()` on the server socket with timeout 0. Accept if readable. Send `ReadyMessage` to the new client.

2. **Poll clients.** `SocketServer.poll_clients()` runs `select()` on all client sockets with timeout 0. Read available data into the per-client `FrameReader`. Dispatch complete messages to `DisplayServer._handle_message()`, which routes by `isinstance` to the per-message-type handler. Remove errored sockets via `remove_client()`.

3. **Render scene.** Enable docking via `dock_space_over_viewport`. Render the idle flame background (`idle_screen.render_idle()`). Check for World menu background click. Render World panel if open. Render all workspace frames (see §5). Render the dock bar for minimized frames. Legacy fallback: if no frames exist, render unframed scenes as tabs.

4. **Flush events.** Broadcast all buffered `InteractionMessage`s via `SocketServer.send_to_client()` to every connected client, then clear the queue. Each event is also recorded in `QueryDispatcher._recent_events` for later `list_recent_events()` queries.

All four phases are non-blocking. The frame completes in microseconds when idle. ImGui handles frame pacing via GLFW vsync; `hello_imgui.fps_idling.fps_idle = 30.0` caps the idle frame rate.

4.2 Multi-Scene State

Scene state is owned by `SceneManager` in `src/punt_lux/scene/manager.py`, exposed to the rendering layer through read-only properties on `DisplayServer.scene_manager`.

Field	Type and Purpose
_scenes	dict[str, SceneMessage] — unframed scenes; ordered by insertion; keyed by scene_id.
_scene_order	list[str] — explicit tab ordering for unframed scenes.
_active_tab	str | None — currently selected tab in the unframed-scenes view.
_scene_widget_state	dict[str, WidgetState] — per-scene slider/checkbox/combo/text state.
_dirty_windows	set[str] — window element IDs needing initial position set.
Frame state	
_frames	dict[str, Frame] — all workspace frames, keyed by frame_id.
_focus_frame_id	str | None — frame to auto-focus on next render.
_scene_to_frame	dict[str, str] — scene → frame lookup.
_scene_to_owner	dict[str, int] — scene → client fd ownership.
Display-server state	
_event_queue	list[InteractionMessage] on DisplayServer, drained each frame.
_client_names	On SocketServer, fd → display name from ConnectMessage.
_menu_registrations	On MenuManager, per-client agent menu items.

4.2.1 Scene Lifecycle

New scene (scene_id not in _scenes)
Appended to _scenes and _scene_order. Fresh WidgetState allocated. Becomes the active tab. Window elements marked dirty. Existing events preserved.

Replace-in-place (same scene_id)
Content replaced in _scenes. Widget state and render function state cleared for that scene. Events for elements that existed in the old scene but not the new scene are drained via on_scene_replaced callback (using full subtree ID collection through _collect_ids()). Events from other scenes are untouched.

Dismiss (tab close button)
Scene removed from all state dicts. Window element IDs removed from _dirty_windows. Active tab moves to the adjacent neighbor (next tab, or previous if rightmost was dismissed).

Clear (ClearMessage or menu)
All scenes, widget state, events, and dirty windows cleared. Fresh instances allocated (not .clear() on aliased objects).

4.2.2 Widget State Swap Pattern

The 24-element ElementRenderer._RENDERERS dispatch table all reads from self._widget_state. Rather than threading a scene_id parameter through every method, the display swaps the per-scene state dict into this instance variable before rendering each tab:

```
def _render_scene_tab(self, scene_id: str) -> None:
    self._element_renderer.widget_state = \
        self._scene_manager.scene_widget_state[scene_id]
    self._element_renderer.current_scene_id = scene_id
    scene = self._scene_manager.scenes[scene_id]
    for elem in scene.elements:
        self._element_renderer.render_element(elem)
```

This is the same pattern ImGui itself uses: "current context." Zero changes required in any renderer method.

4.3 Table Filtering

Tables support built-in filtering at frame rate with zero MCP round-trips. `TableRenderer` (in `src/punt_lux/display/tab` renders filter controls (search input, combo dropdowns) above the table and applies filters in the render loop:

1. Filter state stored in `WidgetState` with prefixed keys (e.g., `__tbl_search_{eid}`, `__tbl_filter_{col}_{eid}`).
2. `_apply_table_filters()` renders controls, snapshots state, detects changes.
3. `_filter_indexed_rows()` applies AND logic across all active filters (case-insensitive substring for search, exact match for combos).
4. Pagination resets to page 0 when filters change.
5. Detail panel renders a two-column metadata grid plus long-form body text for the selected row.

5 Frame Architecture

Since v0.9.0, scenes can target named *frames*—persistent ImGui windows that outlive client connections. Frames are the primary unit of workspace organization. The `Frame` class lives in `src/punt_lux/scene/frame.py`; `SceneManager` owns the `_frames` dict and the scene-to-frame routing.

5.1 Frame Class

Field	Type and Purpose
`frame_id`	`str` — Unique identifier.
`title`	`str` — Window title bar text.
`owner_fds`	`set[int]` — File descriptors of contributing clients.
`scenes`	`dict[str, SceneMessage]` — All scenes in this frame, keyed by `scene_id`.
`scene_order`	`list[str]` — Tab ordering.
`active_tab`	`str | None` — Currently visible scene.
`minimized`	`bool` — Docked to bottom bar.
`cascade_index`	`int` — Stagger offset for initial positioning.
`initial_size`	`tuple[int,int] | None` — Width/height hint (first use only).
`flags`	`dict[str,bool] | None` — ImGui window flags (`no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`).
`layout`	`Literal["tab", "stack"]` — Multi-scene composition mode.

Public read/write properties wrap each field (PR #159 extracted `_Frame` from the original `display.py` and renamed it `Frame`; constructor follows the PY-CC-1 `__new__` pattern).

5.2 Frame Lifecycle

Creation
A frame is created on the first `SceneMessage` with a `frame_id` not already in `_frames`. The client's fd is added to `owner_fds`.

Shared ownership
Multiple clients can contribute scenes to the same frame. Each client that sends a scene targeting the frame is added to `owner_fds`.

Orphan model
When a client disconnects, its scenes persist with an `_ORPHAN_FD` sentinel (`-1`) in place of the real fd. The frame remains visible and interactive. A new client connecting with the same `frame_id` adopts the orphaned scenes.

Dismissal
The close button removes the frame and all its scenes from state. Widget state for all scenes in the frame is cleaned up.

Focus management
New frames are positioned using cascade staggering (`cascade_index` increments per frame). Initial size is applied on first render only. `_focus_frame_id` triggers `SetNextWindowFocus` on the next render cycle.

5.3 World Menu

Right-clicking the docking background opens the World menu panel, providing frame management operations: list all frames, restore minimized frames, close all frames, and access display settings.

5.4 Dock Bar

Minimized frames appear as clickable buttons in a bottom dock bar. Clicking a dock bar button restores the frame to its previous position and size.

6 IPC and Process Management

The shipped system runs the display and the session hub in a single process. The target architecture in `x11-model.md` splits this into three processes (mcp-proxy → `luxd` → `lux-display`); `luxd-impl.md` is the Phase 1 implementation spec. This section describes the current single-process reality.

6.1 Socket Discovery

`DisplayPaths` in `src/punt_lux/paths.py` resolves the display socket path in priority order:

1. `$LUX_SOCKET` environment variable.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.
3. `/tmp/lux-$USER/display.sock` (fallback).

The fallback path is chosen to stay within macOS's ~104-character `AF_UNIX` path limit.

The hub socket and PID files are resolved by module-level helpers in the same file: `hub_dir()`, `hub_pid_path()`, `hub_port_path()`, `hub_log_dir()`, `read_hub_port()`, `is_hub_running()`.

6.2 PID Lifecycle

- PID file written at `{socket_path}.pid` on server startup.
- Clients verify liveness via `os.kill(pid, 0)` before connecting.
- Stale socket cleanup: `DisplayPaths.cleanup_stale()` removes socket and PID file if the process is dead.

6.3 Auto-Spawn

`DisplayPaths.ensure_display()` spawns the display server if not running:

1. Checks socket existence and PID liveness.
2. If not running, spawns
   `python -m punt_lux display --socket {path}`
   with `start_new_session=True` (detached from caller's process group).

3. Polls socket existence + PID liveness every 100 ms until ready or 5 s timeout.
4. Display logs to {socket_path}.log.

6.4 MCP Server Reconnect

The MCP server wraps every tool call in tools/connection.py::_with_reconnect(): if an OSError (broken pipe) occurs, the stale client is closed, a new connection is established, and the call is retried exactly once. No retry loop.

7 Performance

7.1 Frame Budget

Parameter	Value	Notes
Idle FPS	30	fps_idling.fps_idle = 30.0
Active FPS	60+	GLFW vsync, GPU-bound
Socket poll timeout	0	Non-blocking select()
Max message size	16 MiB	Hard limit in FrameReader
Connection timeout	5.0 s	ensure_display() and handshake
Texture loading	Lazy	First render triggers PIL + OpenGL upload
Event ring buffer	200 entries	QueryDispatcher._recent_events
Error ring buffer	100 entries	QueryDispatcher._recent_errors

7.2 Critical Path Analysis

The frame-critical path is:

1. select() on N client sockets — $O(N)$, $N < 64$ in practice.
2. FrameReader.feed() — memcpy into buffer, yield frames. $O(B)$ where B is bytes received.
3. DisplayServer._render_scene() — ImGui draw list calls via ElementRenderer.render_element(). $O(E)$ where E is total element count across visible scene.
4. _flush_events() — send K events to N clients. $O(K \cdot N)$.

Table filtering is $O(R \cdot C)$ where R is row count and C is column count. This runs every frame but is fast for tables under ∼10,000 rows. No benchmarks exist for larger tables; pagination (configurable page size) bounds visible row rendering.

7.3 Idle Screen

The idle screen renders ∼60 draw calls per frame: 48 radial lines, 3 glow circles, 3 flame shapes (each ∼7 bezier operations). This is negligible for a GPU-rendered immediate-mode UI at 30 FPS idle.

7.4 Memory

- **Scene storage.** Scenes are Python dataclass trees held in memory. No size limit beyond the 16 MiB message cap. Each scene is a separate dict entry; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs held in TextureCache until _on_exit(). No eviction policy. Image-heavy scenes accumulate GPU memory.
- **Widget state.** Per-scene WidgetState is a flat dict. Cleared on scene replacement. No growth concern.
- **Event queue.** Flushed every frame. Bounded by interaction rate (<100 events/frame in practice).
- **Ring buffers.** deque(maxlen=200) for events and deque(maxlen=100) for errors; constant memory.

7.5 Known Limitations

1. **No ImGui ID scoping.** Widget IDs are derived from `element_id` alone, not scoped by scene. ImGui internal state (focus, open/closed) can bleed between tabs with colliding IDs.
2. **No texture eviction.** Long-running sessions with many images accumulate GPU memory.
3. **Filter is per-frame.** Table filtering re-executes every frame even when nothing changed. Not a problem at current scale but would benefit from dirty-flag optimization for 10,000+ row tables.

8 Security

8.1 Threat Model

The primary trust boundary is the Unix socket. The display server accepts connections from any process that can reach the socket file. In the default configuration (`/tmp/lux-$USER/`), the socket is protected by filesystem permissions (user-only directory, mode `0700`).

Threat	Description	Mitigation
Socket hijacking	Another process connects to the socket and injects scenes or reads events.	Directory permissions (0700) on `/tmp/lux-$USER/`.
Malformed messages	Client sends invalid JSON, unknown message type, or oversize payload.	Unknown types deserialized as `UnknownMessage`; oversize payloads rejected by FrameReader.
Render function	Agent-submitted Python code executes arbitrary operations.	Consent dialog with AST warnings. User must explicitly click Allow.
Resource exhaustion	Scene with millions of elements or very large images.	16 MiB message cap. No element count limit (ImGui handles gracefully).

8.2 Render Function Security

Render functions are the only element kind that executes arbitrary code. The security model is **consent-based, not sandbox-based**:

1. **AST scanning**: a best-effort visitor flags suspicious imports (`os`, `subprocess`, `socket`, `ctypes`, `urllib`, etc.) and dangerous builtins (`eval`, `exec`, `open`, `__import__`). This is a *UX signal*, not a security boundary.
2. **Consent dialog**: an ImGui modal displays the full source code with line numbers and any AST warnings. The user must click "Allow" to proceed. Dismissing the modal (Esc) is treated as "Deny." Consent is per-element: changing the source code resets consent.
3. **Execution**: `CodeExecutor` compiles the source and calls `render(ctx)` each frame. The `RenderContext` provides a persistent state dict, delta time, frame counter, canvas dimensions, and a `send()` callback for emitting events. Execution is **not sandboxed**: the code runs in the display server's process with full access to the Python runtime. Runtime errors are caught and displayed, not propagated.

Implication: render functions should be treated as user-approved code, equivalent to running a script the user downloaded. The consent dialog provides informed consent, not isolation.

8.3 Protocol Robustness

- **Malformed JSON**: client disconnected, not crashed. `SocketServer` logs the error via the `on_error` callback and removes the client.
- **Unknown message type**: deserialized as `UnknownMessage` for forward compatibility.

- **Oversize frame**: `FrameReader` checks `payload_len` against `MAX_MESSAGE_SIZE` and raises; `SocketServer` converts to disconnect.
- **Partial reads**: `FrameReader` accumulates bytes across frames; no assumption of complete messages per `recv()`.
- **Identity fields protected**: `SceneManager.apply_update()` silently drops patches that attempt to change `id` or `kind` on an element.
- **Double-remove idempotent**: `SocketServer.remove_client()` is safe to call twice on the same socket.

9 Platform Integration

9.1 macOS

- **Dock presence.** The display server runs as a normal macOS Dock app per GLFW's default activation policy (Regular). The Dock icon and Cmd-Tab entry are intentionally visible to aid operational debugging.
- **Process name.** `setproctitle("Lux")` makes the process identifiable in `ps` and `top`. Does not affect Activity Monitor's binary name column.
- **Font discovery.** Prefers Arial Unicode or Helvetica; merges Apple Symbols for mathematical/Z notation glyphs and STIX Two Math for Mathematical Alphanumeric Symbols (U+1D400–1D7FF).
- **AF_UNIX path limit.** macOS limits Unix socket paths to ∼104 characters. `tempfile.mkdtemp(prefix="lux-")` used in tests; production paths are short by design.

9.2 Linux

- **Font discovery.** DejaVu Sans or Noto Sans; merges Noto Sans Symbols2 and Noto Sans Math for Unicode coverage.
- **XDG compliance.** Prefers `$XDG_RUNTIME_DIR/lux/` for socket placement.
- **No Dock equivalent.** Window managers use the X11 window title (set to "Lux").

10 MCP Tool Surface

24 tools, registered in `src/punt_lux/tools/tools.py`. 16 are declared with `@mcp.tool()` directly; 8 are declared with the `@_query_tool(method=...)` wrapper that routes through the generic `QueryRequest` envelope (see `introspection-api.md`).

Tool	Signature and Purpose
Scene management	
`show`	`(scene_id, elements, title?, layout?, frame_id?, frame_title?, frame_size?, frame_flags?)` — Display a scene. Docstring lists all 24 element kinds.
`show_table`	`(scene_id, columns, rows, filters?, detail?, title?)` — Filterable data table with search, combo filters, and detail panel.
`show_dashboard`	`(scene_id, metrics?, charts?, table_columns?, table_rows?, title?)` — Metrics cards + charts + summary table.

Tool	Signature and Purpose
update	(scene_id, patches) — Incremental JSON patches targeting elements by ID.
clear	() — Clear all scenes.
Communication	
ping	() — Latency probe; returns PongMessage.
recv	(timeout?) — Receive next event (interaction, window lifecycle).
set_menu	(menus) — Set agent-provided menu bar entries.
register_tool	(name, description, schema) — Register a custom tool in the display's tool palette.
set_theme	(theme) — Apply display theme. Routes through QueryRequest.
Configuration	
display_mode	() — Read current display mode (y/n); advisory L3 signal for consumer plugins.
set_display_mode	(mode) — Set display mode.
set_window_settings	(opacity?, font_scale?, decorated?, fps_idle?) — Configure display window appearance and idle frame rate.
set_frame_state	(frame_id, minimized?) — Control frame state (minimize/restore).
Introspection (legacy ad-hoc messages)	
inspect_scene	(scene_id) — Element tree for a scene. Implementation routes through the generic QueryRequest envelope; the IntrospectRequest message type remains in the protocol for backward compatibility.
list_scenes	() — All active scenes plus frame summaries. Same dual-path treatment as inspect_scene.
screenshot	() — Capture display as a PNG; returns the file path.
Introspection (generic query path)	
get_display_info	() — Display dimensions, frame count, client count, FPS, uptime, PID.
get_window_settings	() — Opacity, font scale, decorated, idle FPS.
get_theme	() — Current theme and available themes.
list_clients	() — Connected clients with display names and connect times.
list_menus	() — Registered menu items, per client.
list_recent_events	(count?) — Recent interaction events. Reads QueryDispatcher._recent_events (max 200).
list_errors	(count?) — Recent error log entries. Reads QueryDispatcher._recent_errors (max 100).

The introspection-api proposal targets fourteen query methods (see introspection-api.md Section 2 for the full inventory). Twelve are wired and live; screenshot is registered but its handler raises and routes to the dedicated ScreenshotRequest path; one op from the inventory remains unimplemented.

Six are built into QueryDispatcher at construction time: inspect_scene, list_scenes, list_clients, list_menus, list_recent_events, list_errors.

Seven more handlers are registered by DisplayServer at construction time because they touch ImGui state: screenshot (stub — raises), get_display_info, get_window_settings, get_theme, set_window_settings, set_frame_state, set_theme.

11 Formal Model

The display server's state machine is formally specified in `docs/architecture/display-server.tex` using the Z notation (ISO 13568). The specification covers:

- Client connection/disconnection with FrameReader association.
- Scene replacement, incremental patching, and clearing.
- Interaction event generation and broadcast.
- FrameReader streaming buffer invariants.

The specification has been type-checked with Spivey's `fuzz` and model-checked with ProB (7,943 reachable states explored). Refinement tests in `tests/test_display_refinement.py` verify commutativity between the abstract Z operations and the concrete Python implementation via an abstraction function (`tests/display_abstraction.py`).

Partition tests in `tests/test_display_partition.py` derive test cases from the Z specification using the Test Template Framework (TTF), covering all input space partitions of each operation schema.

Note: the formal model predates multi-scene tabs and models a single-scene display. The abstraction function maps multi-scene state to the active tab for backward compatibility. A specification update to model the full multi-scene state is pending.

12 Domain Model Gap

The companion document `docs/architecture/domain-model.md` names the north-star OO domain — Element, Scene, Client, Update, Event, Port — with structural invariants (identity unique within a Scene; ownership immutable; clients mutate only what they own; acyclic tree; typed property updates; transactional disconnection).

The shipped system implements a subset of those invariants in procedural form. Specifically:

- Elements are `frozen=True` dataclasses with public fields. The target makes them stateful objects with validated mutators.
- Updates are coarse: `SceneMessage` replaces the whole tree; `UpdateMessage` carries a list of patches that the `SceneManager` applies imperatively. The target decomposes into five typed semantic primitives (`AddElement`, `RemoveElement`, `SetProperty`, `ReparentElement`, `ReplaceElement`).
- Ownership is tracked in `SceneManager._scene_to_owner` by client fd. Cross-client mutations within the same scene are not currently rejected. The target enforces ownership at the domain layer.
- Events are limited to `InteractionMessage` (input callbacks). The target emits success and failure events for every domain mutation, replayable by Port subscribers.

The draw-command surface (PR #176, #177) is the first piece that moves toward the north star: typed value classes (`Point2`, `Color`, `Thickness`, `Radius`, `Rounding`), decoder at the wire boundary, records that own their `to_wire` / `from_wire`. The protocol-codec split into family modules with `ElementCodec` and `MessageRegistry` (PR #157, #169, #170, #172) is the second. The remaining work is tracked as bead `lux-x4kb`: per-kind codecs still live as module-level functions rather than methods on the dataclasses.

The OO ratchet (`make check-oo`) enforces the local rule that every commit improves OO scores on touched files. Aggregate scores as of v0.18.0: `method_ratio` 0.68 (target $\geq$ 0.80), `max_complexity` 19 (target $\leq$ 10), `module_size` ~1,400 max (target $\leq$ 300) — `display/server.py` is the largest remaining decomposition target. See `docs/oo-refactor/resume.md` for the per-file breakdown.

13 Test Architecture

The default `pytest` run collects 853 tests (out of 869 collected; 16 marker-gated tests are deselected). Counts below are from `pytest --collect-only`.

Test File	Count	What It Tests
`test_protocol.py`	202	Serialization, deserialization, framing
`test_display_partition.py`	107	TTF-derived partitions from Z spec
`test_tools.py`	83	MCP tool behavior (mocked client)
`test_display_state.py`	67	State machine logic (no ImGui)
`test_display_refinement.py`	28	Abstraction function commutativity
`test_show.py`	29	CLI `show` subcommand
`test_draw_commands.py`	27	Typed draw-command records
`test_display_client.py`	26	High-level client library
`test_draw_decoder.py`	24	Draw-command dispatch on wire `cmd`
`test_draw_values.py`	24	Point2/Color/Thickness validation
`test_query.py`	23	Query envelope round-trips
`test_draw_wire.py`	23	WireContext require-helpers
`test_scene_manager.py`	23	SceneManager state transitions
`test_runtime.py`	22	CodeExecutor lifecycle
`test_paths.py`	19	Socket discovery, PID, auto-spawn
`test_hooks.py`	18	Hook handler output
`test_draw_bounds.py`	15	Radius/Rounding bounds
`test_config.py`	12	Config read/write
`test_remote.py`	12	mcp-proxy and remote transport
`test_cli.py`	11	CLI command smoke tests
`test_paths_hub.py`	11	Hub socket and PID paths
`test_service.py`	11	Daemon lifecycle
`test_unit.py`	6	Misc unit tests
`test_hub.py`	6	WebSocket session hub
`test_menu_manager.py`	6	MenuManager registration
`test_table_renderer.py`	6	Table renderer pure logic
`test_query_dispatcher.py`	5	QueryDispatcher routing
`test_socket_server.py`	5	SocketServer non-blocking IO
`test_version.py`	2	Version string consistency
Marker-gated (not in default `pytest` run)		
`test_integration.py`	8	Socket IPC, protocol framing (real sockets)
`test_query.py` (integration)	5	Query end-to-end with real socket
`test_e2e.py`	3	Full display server lifecycle (requires GPU)
Total	869	(853 default + 16 marker-gated)

Quality gates: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest` (all markers except integration/e2e), `check-oo` (OO ratchet against `.oo-baseline.json`), `check-suppressions` (suppression ratchet). Zero violations required. See `Makefile` for the exact targets; `make check` is the composite gate.